```c
#include <stdio.h>

int reverseNum(int N) {

        // Function to store the reversed number
    int rev = 0;
    while (N > 0) {

        // Extract the last digit
        int dig = N % 10;

        // Append the digit to the reversed number
        rev = rev * 10 + dig;

        // Remove the last digit
        N /= 10;
    }
    return rev;
}
```

```c
int isPalindrome(int N) {

    return N == reverseNum(N);
}

int main() {
    int N = 121;
    if (isPalindrome(N)) {
        printf("Yes\n");
    }
    else {
        printf("No\n");
    }
    return 0;
}
```

TEST CASE ID	TEST SCENARIO	TEST STEPS	TEST DATA	EXPECTED OUTPUT	ACTUAL OUTPUT	STATUS
TC01	ValidPalindrome Number	Enter the value	121	Yes	Yes	Pass
TC02	Non-Palindrome Number	Enter the value	123	No	No	Pass
TC03	Single Digit Number	Enter the value	7	Yes	Yes	Pass
TC04	Two digit Palindrome	Enter the value	11	Yes	Yes	Pass
TC05	Two digit non-Palindrome	Enter the value	12	No	No	Pass
TC06	Number ending with zero	Enter the value	10	No	No	Pass
TC07	Large Palindrome Number	Enter the value	12321	Yes	Yes	Pass
TC08	Palindrome number with even digits	Enter the value	2442	Yes	Yes	Pass
TC09	Palindrome Number with reverse logic missing	Enter the value	1331	Yes	No	Pass
TC10	Negative Number	Enter the value	-121	No	Yes	Fail

```c
// C program to find roots of
// a quadratic equation
#include <math.h>
#include <stdio.h>
#include <stdlib.h>

// Prints roots of quadratic
// equation ax*2 + bx + x
void findRoots(int a, int b, int c)
{
    // If a is 0, then equation is
    // not quadratic, but linear
    if (a == 0) {
        printf("Invalid");
        return;
    }

    int d = b * b - 4 * a * c;
    double sqrt_val = sqrt(abs(d));

    if (d > 0) {
        printf("Roots are real and different\n");
        printf("%f\n%f", (double)(-b + sqrt_val) / (2 * a),
               (double)(-b - sqrt_val) / (2 * a));
    }
  else if (d == 0) {
        printf("Roots are real and same\n");
        printf("%f", -(double)b / (2 * a));
    }
    else // d < 0
    {
        printf("Roots are complex\n");
        printf("%f + i%f\n%f - i%f", -(double)b / (2 * a),
               sqrt_val / (2 * a), -(double)b / (2 * a),
               sqrt_val / (2 * a));
    }
}

// Driver code
int main()
{
    int a = 1, b = -7, c = 12;

    // Function call
    findRoots(a, b, c);
    return 0;
}
```

TEST CASE ID	TEST SCENARIO	TEST STEPS	TEST DATA	EXPECTED OUTPUT	ACTUAL OUTPUT	STATUS
TC01	Real & Distinct Roots	Enter the value	1, -7, 12	Roots are real and different: 4.0, 3.0	Roots: 4.0, 3.0	PASS
TC-02	Real & Same Roots	Enter the value	1, -2, 1	Roots are real and same: 1.0	Roots: 1.0	PASS
TC-03	Complex Roots	Enter the value	1,2,5	Roots are complex: -1.0 + i2.0, -1.0 - i2.0	-1.0 ± i2.0	PASS
TC-04	Not a Quadratic (a=0)	Enter the value	0,5,10	"Invalid"	"Invalid"	PASS
TC-05	Large Coefficients (Overflow)	Enter the value	30000,30000,1	Roots are real and different	Program Crashes or garbage values	FAIL
TC-06	All Zeros	Enter the value	0,0,0	"Invalid"	"Invalid"	PASS
TC-07	Floating Point Inputs	Enter the value	1.5,3.5,2	Roots: -1.0, -1.33	Truncated to 1, 3, 2 (as code uses `int`)	FAIL
TC-08	Negative Coefficients	Enter the value	-1, 5, -6	Roots: 2.0, 3.0	Roots: 2.0, 3.0	PASS
TC-09	Very Small Discriminant	Enter the value	1, 2.00001, 1	Roots are real and different	Treated as "Real and same" due to int rounding	FAIL
TC-10	Extreme Negative a	Enter the value	-100, 10, 1	Roots are real and different	Roots: -0.05, 0.15	PASS

```c
#include <stdio.h>

void processTransaction(double *balance, char type, double amount) {
    if (amount <= 0) {
        printf("Error: Amount must be positive.\n");
        return;
    }

    if (type == 'd' || type == 'D') {
        *balance += amount;
        printf("Deposited: $%.2f. New Balance: $%.2f\n", amount, *balance);
    }
    else if (type == 'w' || type == 'W') {
        if (amount > *balance) {
            printf("Error: Insufficient funds. Balance: $%.2f\n", *balance);
        } else {
            *balance -= amount;
            printf("Withdrawn: $%.2f. New Balance: $%.2f\n", amount, *balance);
        }
    }
    else {
        printf("Error: Invalid transaction type.\n");
    }
}

int main() {
    double myBalance = 500.00;
    processTransaction(&myBalance, 'd', 200.0);
    processTransaction(&myBalance, 'w', 800.0);
    processTransaction(&myBalance, 'w', 150.0);
    return 0;
}
```

Test ID	Test Scenario	TEST STEPS	TEST DATA	EXPECTED OUTPUT	ACTUAL OUTPUT	STATUS
TC-01	Valid Deposit	Type 'd', amount 200	bal=500, amt=200	New Balance: 700.00	New Balance: 700.00	PASS
TC-02	Valid Withdrawal	Type 'w', amount 150	bal=700, amt=150	New Balance: 550.00	New Balance: 550.00	PASS
TC-03	Insufficient Funds	Withdraw > Balance	bal=550, amt=800	Error: Insufficient funds	Error: Insufficient funds	PASS
TC-04	Case Sensitivity	Use 'D' instead of 'd'	bal=550, amt=50	New Balance: 600.00	New Balance: 600.00	PASS
TC-05	Negative Amount	Enter -100 deposit	bal=600, amt=-100	Error: Amount must be positive	Error: Amount must be positive	PASS
TC-06	Invalid Type	Enter 'x' for type	bal=600, type='x'	Error: Invalid transaction type	Error: Invalid transaction type	PASS
TC-07	Zero Amount	Enter 0.00	bal=600, amt=0	Error: Amount must be positive	Error: Amount must be positive	PASS
TC-08	Precision Handling	Small decimal deposit	bal=100, amt=0.001	New Balance: 100.00 (rounded)	New Balance: 100.00	PASS
TC-09	Floating Point Error	Multiple small decimals	Repeat .10 deposits	Exact balance match	*Potential rounding drift*	FAIL
TC-10	Massive Withdrawal	Amt > Double limit	bal=100, amt=1e309	Error: Insufficient funds	Overflows to Infinity	FAIL

```c
#include <stdio.h>

void classifyTriangle(int s1, int s2, int s3) {
    // 1. Check if the sides can actually form a triangle
    // Triangle Inequality Theorem: sum of two sides > third side
    if (s1 + s2 <= s3 || s1 + s3 <= s2 || s2 + s3 <= s1) {
        printf("Error: Not a valid triangle.\n");
        return;
    }

    // 2. Classify based on side lengths
    if (s1 == s2 && s2 == s3) {
        printf("Result: Equilateral Triangle\n");
    }
    else if (s1 == s2 || s2 == s3 || s1 == s3) {
        printf("Result: Isosceles Triangle\n");
    }
    else {
        printf("Result: Scalene Triangle\n");
    }
}

int main() {
    int side1 = 5, side2 = 5, side3 = 8;
    printf("Sides: %d, %d, %d\n", side1, side2, side3);
    classifyTriangle(side1, side2, side3);
    return 0;
}
```

Test ID	Test Scenario	TEST STEPS	TEST DATA	EXPECTED OUTPUT	ACTUAL OUTPUT	STATUS
TC-01	All sides equal	Enter the values	5, 5, 5	Equilateral Triangle	Equilateral Triangle	PASS
TC-02	Two sides equal	Enter the values	5, 5, 8	Isosceles Triangle	Isosceles Triangle	PASS
TC-03	No sides equal	Enter the values	3, 4, 5	Scalene Triangle	Scalene Triangle	PASS
TC-04	Invalid: Sum of 2 = 3rd	Enter the values	2, 2, 4	Not a valid triangle	Not a valid triangle	PASS
TC-05	Invalid: Sum of 2 < 3rd	Enter the values	`1, 2, 10`	Not a valid triangle	Not a valid triangle	PASS
TC-06	Zero side length	Enter the values	0, 5, 5	Not a valid triangle	Not a valid triangle	PASS
TC-07	Negative side length	Enter the values	`-5, 5, 5`	Not a valid triangle	Not a valid triangle	PASS
TC-08	Decimal side inputs	Enter the values	`3.5, 3.5, 3.5`	Equilateral Triangle	*Truncated to 3,3,3 (Equilateral)*	FAIL
TC-09	Large Integer Overflow	Enter the values	2e9, 2e9, 2e9	Equilateral Triangle	*Negative sum (Overflow)*	FAIL
TC-10	Non-numeric input	Enter the values	"a", 5, 5	Error Message	*Program Crash/Garbage*	FAIL

```c
#include <stdio.h>
#include <string.h>

struct Student {
    char name[50];
    int score;
    char grade;
};

void calculateGrade(struct Student *s) {
    if (s->score >= 90) s->grade = 'A';
    else if (s->score >= 80) s->grade = 'B';
    else if (s->score >= 70) s->grade = 'C';
    else if (s->score >= 60) s->grade = 'D';
    else s->grade = 'F';
}

void displayReport(struct Student students[], int count) {
    if (count <= 0) {
        printf("Error: No student records found.\n");
        return;
    }

    float total = 0;
    int topIndex = 0;

    printf("\n--- Student Grade Report ---\n");
    for (int i = 0; i < count; i++) {
        calculateGrade(&students[i]);
        total += students[i].score;

        if (students[i].score > students[topIndex].score) {
            topIndex = i;
        }

        printf("Name: %-10s | Score: %3d | Grade: %c\n",
               students[i].name, students[i].score, students[i].grade);
    }

    printf("-------------------------- \n");
    printf("Class Average: %.2f\n", total / count);
    printf("Top Performer: %s with %d\n", students[topIndex].name,
students[topIndex].score);
}

int main() {
    struct Student classList[3] = {
        {"Alice", 92, ' '},
        {"Bob", 78, ' '},
        {"Charlie", 55, ' '}
    };

    displayReport(classList, 3);
    return 0;
}
```

Test ID	Test Scenario	TEST STEPS	TEST DATA	EXPECTED OUTPUT	ACTUAL OUTPUT	STATUS
TC-01	Grade 'A' Boundary	Enter the value	Score = 90	Grade: A	Grade: A	PASS
TC-02	Grade 'F' Boundary	Enter the value	Score = 59	Grade: F	Grade: F	PASS
TC-03	Average Calculation	Enter the value	Scores: 90, 80, 70	Average: 80.00	Average: 80.00	PASS
TC-04	Top Performer Tie	Enter the value	Alice: 95, Bob: 95	Top: Alice (First found)	Top: Alice	PASS
TC-05	Empty List	Enter the value	count = 0	Error: No records found	Error: No records found	PASS
TC-06	Score > 100	Enter the value	Score = 150	Grade: A	Grade: A	PASS
TC-07	Negative Score	Enter the value	Score = -10	Grade: F	Grade: F	PASS
TC-08	Long Name Overflow	Enter the value	Name = "A-very-long-name..."	Buffer Overflow / Crash	System Crash	**FAIL**
TC-09	Large Score Overflow	Enter the value	Score = 2,147,483,648	Grade: F (due to wrap)	Grade: F	**FAIL**
TC-10	Floating Point Score	Enter the value	Score = 89.9	Truncated to 89 (Grade B)	Grade: B	**FAIL**

control flow diagram code

```
#include <stdio.h>

int reverseNum(int N) {

// 6
int rev = 0;

// 7
while (N > 0) {

// 8
int dig = N % 10;

// 9
rev = rev * 10 + dig;

// 10
N /= 10;
}

// 11
return rev;
}

int isPalindrome(int N) {

// 12
int r = reverseNum(N);

// 13
if (N == r)
return 1;
else
return 0;
}

int main() {

// 1
int N = 121;

// 2
int result;

// 3
result = isPalindrome(N);

// 4
if (result == 1)

// 14
printf("Yes\n");

else

// 15
printf("No\n");

// 5
return 0;
```

Control Flow Diagram